

Transaction and Optimistic Concurrency Control

In this chapter, you will implement all interfaces of `Transaction`. Your implementation will maintain a private workspace for modifications inside a transaction, and commit them in batch, so that all modifications within the transaction will only be visible to the transaction itself until commit. We only check for conflicts (i.e., serializable conflicts) when commit, and this is optimistic concurrency control.

To run test cases,

```
cargo x copy-test --week 3 --day 5
cargo x scheck
```

Task 1: Local Workspace + Put and Delete

In this task, you will need to modify:

```
src/mvcc/txn.rs
```

You can now implement `put` and `delete` by inserting the corresponding key/value to the `local_storage`, which is a skiplist memtable without key timestamp. Note that for deletes, you will still need to implement it as inserting an empty value, instead of removing a value from the skiplist.

Task 2: Get and Scan

In this task, you will need to modify:

```
src/mvcc/txn.rs
```

For `get`, you should first probe the local storage. If a value is found, return the value or `None` depending on whether it is a deletion marker. For `scan`, you will need to implement a `TxnLocalIterator` for the skiplist as in chapter 1.1 when you implement the iterator for a memtable without key timestamp. You will need to store a `TwoMergeIterator<TxnLocalIterator, FusedIterator<LsmIterator>>` in the `TxnIterator`. And, lastly, given that the `TwoMergeIterator` will retain the deletion markers in the child

iterators, you will need to modify your `TxnIterator` implementation to correctly handle deletions.

Task 3: Commit

In this task, you will need to modify:

```
src/mvcc/txn.rs
```

We assume that a transaction will only be used on a single thread. Once your transaction enters the commit phase, you should set `self.committed` to true, so that users cannot do any other operations on the transaction. Your `put`, `delete`, `scan`, and `get` implementation should error if the transaction is already committed.

Your commit implementation should simply collect all key-value pairs from the local storage and submit a write batch to the storage engine.

Task 4: Atomic WAL

In this task, you will need to modify:

```
src/wal.rs
src/mem_table.rs
src/lsm_storage.rs
```

Note that `commit` involves producing a write batch, and for now, the write batch does not guarantee atomicity. You will need to change the WAL implementation to produce a header and a footer for the write batch.

The new WAL encoding is as follows:

HEADER		BODY				
FOOTER						
u32	u16	var	u64	u16	var	...
u32						
batch_size	key_len	key	ts	value_len	value	more key-value pairs
...	checksum					

`batch_size` is the size of the `BODY` section. `checksum` is the checksum for the `BODY` section.

There are no test cases to verify your implementation. As long as you pass all existing test cases and implement the above WAL format, everything should be fine.

You should implement `Wal::put_batch` and `MemTable::put_batch`. The original `put` function should treat the single key-value pair as a batch. That is to say, at this point, your `put` function should call `put_batch`.

A batch should be handled in the same mem table and the same WAL, even if it exceeds the mem table size limit.

Test Your Understanding

- With all the things we have implemented up to this point, does the system satisfy snapshot isolation? If not, what else do we need to do to support snapshot isolation? (Note: snapshot isolation is different from serializable snapshot isolation we will talk about in the next chapter)
- What if the user wants to batch import data (i.e., 1TB?) If they use the transaction API to do that, will you give them some advice? Is there any opportunity to optimize for this case?
- What is optimistic concurrency control? What would the system be like if we implement pessimistic concurrency control instead in Mini-LSM?
- What happens if your system crashes and leave a corrupted WAL on the disk? How do you handle this situation?
- When you commit the txn, is it necessary to put everything into the memtable in batch, or you can simply put it key by key? Why?

Bonus Tasks

- **Spill to Disk.** If the private workspace of a transaction gets too large, you may flush some of the data to the disk.

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/mini-lsm.

mini-lsm-book © 2022-2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.